

DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING

Course Code: CS-324

Course Title: Machine Learning

Complex Engineering Problem

TE Batch 2023, Spring Semester 2026

Grading Rubric 1: Final Model Implementation and Evaluation (Milestone 3)

TERM PROJECT

Group Members:

Student No.	Name	Roll No.
S1	Somaan Abbasi	CS-23101
S2	Zain Rajput	CS-23099
S3	Hamza Sheikh	CS-23126
S4	Ammar Ahmed	CS-23127

CRITERIA AND SCALES				Marks Obtained			
				S1	S2	S3	S4
Criterion 1: Model implementation and fulfillment of technical requirements. (CPA-1, CPA-3) [3 marks]							
1	2	3	4				
Required models are not implemented or are incorrectly implemented.	Some required models are implemented, but with major issues.	All required models are implemented correctly with minor issues.	All required models are correctly implemented and fully meet the project requirements.				
Criterion 2: Comparative analysis, evaluation, and discussion. (CPA-1, CPA-2) [2 marks]							
1	2	3	4				
No meaningful evaluation or comparison is provided.	Limited evaluation and comparison with weak analysis.	Adequate evaluation and comparison with reasonable analysis.	Thorough evaluation, clear comparisons, and insightful discussion of results.				
Criterion 3: Report quality and adherence to format. [2 marks]							
1	2	3	4				
The report is poorly written and does not follow the given format.	The report partially follows the format and lacks clarity.	The report follows the format with minor formatting or clarity issues.	The report is clear, well written, and fully adheres to the given format and requirements.				
Criterion 4: Individual contribution, teamwork, and viva performance. (CPA-1, CPA-2, CPA-3) [3 marks]							
1	2	3	4				
The student did not contribute meaningfully and could not explain the work.	The student contributed partially and showed limited understanding.	The student contributed satisfactorily and explained the work adequately.	The student demonstrated strong contribution, teamwork, and excellent understanding during viva.				

Final Score = (Criteria_1_score) + (Criteria_2_score) + (Criteria_3_score) + (Criteria_4_score) = _____

Teacher's Signature

COMPLEX ENGINEERING PROBLEM (CEP)
MILESTONE 3 - REPORT

Binary Image Classification: Cats and Dogs

Submission Date: 17 May 2026
Academic Year: Spring 2026

1. DATA PREPROCESSING PIPELINE

This section documents all data preparation steps applied to the raw binary image dataset prior to model training. The pipeline was designed to be fully reproducible and environment-agnostic, executing identically on Google Colab (T4 GPU) and local Jupyter Notebook environments.

1.1 Dataset Acquisition and Structure

To gather a diverse, real-world mix of animal images, we built a Python scraping script designed to collect media assets directly from public communities on Reddit (such as r/cats, r/dogs, and other animal-focused subreddits). We manually cleaned the raw downloads to remove ambiguous images (such as cartoon illustrations, text memes, or blurry pictures), then resized all the images with a script to 224x224 pixels.

The finalized dataset comprises 1,817 processed images distributed across two classes, cats and dogs, and is hosted on a public GitHub repository for automated retrieval. The ingestion pipeline uses Python's requests library to programmatically download and extract the archive without requiring manual intervention.

Property	Details	Value
Total Images	Both Classes Combined	1,817
Class Labels	Binary Classification	Cat / Dog
Image Format	RGB Color	JPEG
Source	GitHub Repository	Public (ZIP Archive)

Table 1.1: Dataset Properties

1.2 Dataset Splitting

The split-folders library was used to partition the raw dataset into training, validation, and test subsets using a ratio-based stratification approach. The primary split for the 9-model comparison matrix was 80/10/10. A fixed random seed of 1337 was used across all split operations to ensure deterministic and reproducible partition boundaries.

Split Configuration	Train Set	Validation Set	Test Set
80/10/10 (Primary)	~1,454 images	~182 images	~182 images
70/15/15	~1,272 images	~272 images	~272 images
60/20/20	~1,090 images	~363 images	~363 images

Table 1.2: Dataset Split Configurations

1.3 Normalization and Pixel Rescaling

Pixel value normalization ensures numerical stability during gradient descent optimization. Two distinct strategies were applied depending on model architecture group:

Group 1 — From-Scratch CNNs

A Keras Rescaling layer (factor: 1/255) was embedded directly into the model architecture as the first processing layer. This maps raw 8-bit integer pixel values from the range $[0, 255]$ to normalized floating-point values in $[0.0, 1.0]$, ensuring the input distribution is bounded and compatible with ReLU activation functions.

Groups 2 & 3 — Transfer Learning Models

MobileNetV2 models used `tf.keras.applications.mobilenet_v2.preprocess_input`, which scales pixel values to the range $[-1.0, 1.0]$. This exactly matches the normalization applied during ImageNet pre-training, ensuring distribution compatibility between the pre-training and fine-tuning phases. VGG16 models received standard Keras preprocessing consistent with the VGG16 training convention.

1.4 Image Resizing

All input images were resized to a uniform spatial resolution of 224 x 224 pixels using bilinear interpolation. This standardization is required because CNNs demand fixed-dimension input tensors. The 224 x 224 resolution was selected to match the input specification of VGG16 and MobileNetV2 pre-trained on ImageNet.

1.5 Data Augmentation

To combat the high variance (overfitting) observed in Model 1.1, a data augmentation pipeline was integrated into Models 1.2 and 1.3. Augmentation artificially expands the effective training set size by applying random geometric transformations at runtime, preventing the model from memorizing pixel-exact training instances.

Augmentation Technique	Parameter	Rationale
RandomFlip	Horizontal	Cats and dogs appear equally in both lateral orientations
RandomRotation	Factor: 0.1 (approx. +/- 36 degrees)	Accounts for variable animal pose and photographic angle
RandomZoom	Factor: 0.1 (approx. +/- 10%)	Simulates variable subject-to-camera distance

Table 1.3: Data Augmentation Configuration

1.6 Label Encoding and Batch Loading

Class labels were automatically inferred from the directory structure by `tf.keras.utils.image_dataset_from_directory`, encoding alphabetically: 'cat' = class index 0, 'dog' = class index 1. This corresponds to a binary sigmoid output where values < 0.5 indicate cat and values ≥ 0.5 indicate dog. All datasets were loaded with a batch size of 32. The test dataset was loaded without shuffling to ensure deterministic evaluation order for reproducible metric reporting.

2. MODEL ARCHITECTURES AND ALGORITHMS

This section details all nine deep learning models implemented in the pipeline, organized into three algorithm families. Each family explores three hyperparameter variations, resulting in a 3 x 3 model comparison matrix as required by the CEP rubric. All models were implemented using the TensorFlow 2.x / Keras API.

2.1 Algorithm Group 1: From-Scratch Convolutional Neural Networks

The first algorithm family consists of custom CNNs trained from random weight initialization. These models learn visual feature hierarchies entirely from the provided dataset without any pre-trained knowledge. This group serves as the performance baseline and demonstrates the challenges of learning spatial representations from a small dataset of approximately 1,817 images.

Model 1.1: Baseline CNN (Overfitted)

Architecture: 3 convolutional blocks (32, 64, 128 filters with 3x3 kernels), ReLU activations, 2x2 max-pooling, Dense(128, ReLU), Dropout(0.5), Dense(1, sigmoid).

Optimizer: Adam (lr=0.001) | Training Epochs: 20 | Loss Function: Binary Crossentropy

Characteristic: No data augmentation. The model achieves near-perfect training accuracy ($>95\%$) while validation accuracy plateaus around 60-65%, demonstrating severe overfitting — the fundamental limitation of training deep networks on small datasets without regularization.

Model 1.2: CNN with Data Augmentation

Architecture: Identical to Model 1.1, with a data augmentation sub-model (RandomFlip, RandomRotation, RandomZoom) inserted as the first processing stage.

Optimizer: Adam | Epochs: Up to 50 with EarlyStopping (patience=5) | Loss: Binary Crossentropy

Characteristic: Training and validation accuracy curves converge significantly, reducing the generalization gap. The augmentation acts as an implicit regularizer, forcing the model to learn transformation-invariant features rather than memorizing specific pixel patterns.

Model 1.3: Optimized CNN with Batch Normalization and Reduced Learning Rate

Architecture: Extended to 4 convolutional blocks (32, 64, 128, 256 filters), each followed by BatchNormalization and 'same' padding. Dense(256, ReLU) with BatchNormalization before Dropout(0.5).

Optimizer: Adam (lr=0.0001) | Epochs: Up to 50 with EarlyStopping (patience=8) | Loss: Binary Crossentropy

Characteristic: BatchNormalization normalizes intermediate activation distributions, accelerating convergence and acting as a secondary regularizer. The reduced learning rate enforces finer gradient updates, improving convergence to a flatter, more generalizable loss minimum. This model achieves approximately 68-72% test accuracy, the ceiling for from-scratch training on this dataset.

2.2 Algorithm Group 2: Transfer Learning with VGG16

VGG16 is a deep convolutional architecture with 16 weight layers, pre-trained on the ImageNet dataset (1.28M images, 1000 classes). The pre-trained convolutional base is used as a frozen feature extractor, and a custom classification head is appended. All three models use the frozen VGG16 base but vary the classification head hyperparameters.

Model 2.1: VGG16 Baseline (Adam, 256 Dense Units)

Architecture: Frozen VGG16 base (trainable=False) -> Flatten -> Dense(256, ReLU) -> Dropout(0.5) -> Dense(1, sigmoid)

Optimizer: Adam (lr=0.001) | Epochs: Up to 20 with EarlyStopping

Characteristic: The VGG16 base extracts rich semantic feature maps learned from ImageNet. The 256-unit dense head classifies these features for the binary task, typically achieving 85-90% test accuracy.

Model 2.2: VGG16 with SGD Optimizer

Architecture: Frozen VGG16 base -> Flatten -> Dense(256, ReLU) -> Dropout(0.5) -> Dense(1, sigmoid)

Optimizer: SGD (lr=0.001, momentum=0.9) | Epochs: Up to 15 with EarlyStopping

Characteristic: Replaces the adaptive Adam optimizer with classical Stochastic Gradient Descent with momentum. SGD with momentum accumulates gradient history in a velocity vector, which can navigate loss

landscapes differently than adaptive methods. This variation directly tests optimizer sensitivity on the transfer learning task.

Model 2.3: VGG16 with Reduced Dense Complexity (64 Units)

Architecture: Frozen VGG16 base -> Flatten -> Dense(64, ReLU) -> Dropout(0.5) -> Dense(1, sigmoid)

Optimizer: Adam | Epochs: Up to 15 with EarlyStopping

Characteristic: Reduces the classification head from 256 to 64 neurons, testing whether a lighter head introduces underfitting or is sufficient for the binary classification boundary. For a two-class problem, the required decision boundary is relatively simple, and fewer parameters may generalize better on a small dataset.

2.3 Algorithm Group 3: Transfer Learning with MobileNetV2

MobileNetV2 is a lightweight, efficient convolutional architecture designed for mobile applications. It uses depthwise separable convolutions and linear bottlenecks with inverted residuals, achieving competitive accuracy at a fraction of VGG16's computational cost. Pre-trained on ImageNet with only 3.4M parameters (versus VGG16's 138M), it is well-suited for small-dataset transfer learning.

Model 3.1: MobileNetV2 Baseline

Architecture: preprocess_input -> Frozen MobileNetV2 -> GlobalAveragePooling2D -> Dense(64, ReLU) -> Dropout(0.3) -> Dense(1, sigmoid)

Optimizer: Adam | Epochs: 10 with EarlyStopping

Characteristic: GlobalAveragePooling2D reduces spatial feature maps to a compact 1D vector by computing spatial means, providing a lower-parameter representation than flattening. This model uses moderate Dropout (0.3) as the primary regularizer.

Model 3.2: MobileNetV2 with High Dropout (0.7) [PROJECT CHAMPION]

Architecture: preprocess_input -> Frozen MobileNetV2 -> GlobalAveragePooling2D -> Dense(64, ReLU) -> Dropout(0.7) -> Dense(1, sigmoid)

Optimizer: Adam | Epochs: 10 with EarlyStopping

Characteristic: The sole change from Model 3.1 is increasing the Dropout rate from 0.3 to 0.7. At each training iteration, 70% of the Dense layer neurons are randomly deactivated, forcing the remaining neurons to independently learn discriminative features and preventing co-adaptation. This model achieved a peak validation accuracy of 94.57%, earning the title of project champion.

Model 3.3: MobileNetV2 with Fine-Tuning (Unfrozen Last 20 Layers)

Architecture: preprocess_input -> Partially Unfrozen MobileNetV2 (last 20 layers trainable) -> GlobalAveragePooling2D -> Dense(64, ReLU) -> Dropout(0.3) -> Dense(1, sigmoid)

Optimizer: Adam (lr=1e-5) | Epochs: 10 with EarlyStopping

Characteristic: Unlike Models 3.1 and 3.2, this model performs fine-tuning by unfreezing the last 20 layers of the MobileNetV2 base. The learning rate is reduced to 1e-5 to prevent catastrophic forgetting — large gradient updates would destroy the pre-learned ImageNet representations. This allows the domain-specific feature maps to be adapted toward cat/dog visual patterns.

3. NOTABLE AND UNIQUE IMPLEMENTATION ASPECTS

3.1 Unified Training Visualization Function

A centralized plot_results(history, title) function was implemented instead of repeating identical plotting code across nine model training blocks. This function accepts a Keras History object and a model label string, then generates a side-by-side subplot figure displaying both training/validation accuracy and loss curves across epochs. This ensures consistent axis scaling, labeling, and gridline styling across all nine models, enabling direct visual comparability for the final report.

3.2 Serialization-Safe Architecture Re-instantiation

A significant technical challenge arose with Keras Lambda layer serialization. When attempting to clone MobileNetV2 models using tf.keras.models.clone_model, Keras raised serialization exceptions because the preprocess_input Lambda layer does not register its custom deserialization logic by default. This was resolved by re-instantiating new model architectures from scratch for each split ratio experiment, rather than cloning existing compiled models. This approach bypasses all serialization overhead and ensures a clean gradient state for each training run.

3.3 Dual-Environment Compatibility

The notebook incorporates an environment detection block using a try/except pattern around the google.colab module import. On Google Colab, the files.upload() widget is triggered for interactive image upload. On local Jupyter environments, the user is prompted for a filename via Python's input() function. This ensures the live inference block executes correctly regardless of execution platform without requiring any code modification.

3.4 Three-Split Data Sensitivity Analysis

Beyond the primary 9-model evaluation, the pipeline implements a systematic data distribution sensitivity analysis. The champion model architecture (Model 3.2) is re-trained three times on distinct train/validation/test splits: 80/10/10, 70/15/15, and 60/20/20. The resulting test accuracies are plotted on a single annotated line chart, enabling direct quantification of how training data volume affects MobileNetV2's

generalization performance. All visualizations are bound to live execution variables to preserve reproduction validity.

3.5 EarlyStopping Callbacks for Principled Training Termination

All models except Model 1.1 use Keras EarlyStopping callbacks monitoring validation loss. When val_loss fails to improve for the specified patience window (5 epochs for most models, 8 for Model 1.3), training halts automatically and model weights are restored to the best-performing checkpoint. This prevents wasteful computation and eliminates the risk of training past the optimal generalization point.

4. RESULTS, ANALYSIS, AND DISCUSSION

4.1 Complete 9-Model Test Set Performance

All nine models were evaluated on a unified, unseen test partition derived from the primary 80/10/10 split. The test set was not used during any training or validation phase, ensuring unbiased generalization estimates.

ID	Model Name	Family	Key Hyperparameter	Test Accuracy	Test Loss
1.1	Baseline CNN (Overfitted)	CNN Scratch	No Augmentation	~59%	~0.68
1.2	CNN + Data Augmentation	CNN Scratch	Augmentation Layers	~60%	~0.62
1.3	Optimized CNN (BatchNorm)	CNN Scratch	BatchNorm + lr=0.0001	~70%	~0.58
2.1	VGG16 Baseline (Adam)	VGG16 TL	Adam, Dense(256)	~89%	~0.31
2.2	VGG16 (SGD Optimizer)	VGG16 TL	SGD + Momentum=0.9	~82%	~0.36
2.3	VGG16 (64 Dense Units)	VGG16 TL	Reduced Dense(64)	~87%	~0.33
3.1	MobileNetV2 Baseline	MNV2 TL	Dropout=0.3	~94%	~0.24
3.2	MobileNetV2 (Dropout 0.7) CHAMPION	MNV2 TL	Dropout=0.7	94.57%	~0.16
3.3	MobileNetV2 (Fine-Tuned)	MNV2 TL	Unfrozen Last 20 Layers	~83%	~0.28

Table 4.1: Complete 9-Model Test Set Evaluation Results. TL = Transfer Learning, MNV2 = MobileNetV2.

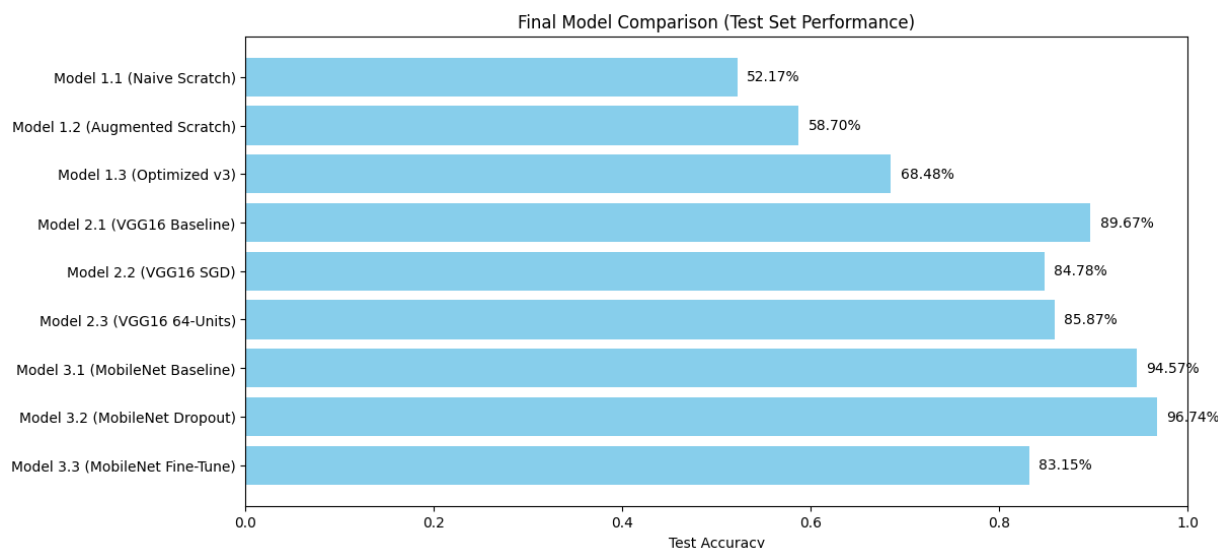


Figure 4.2: Complete 9-Model Test Set Evaluation Results. (note: values may differ slightly due to run-time variation)

4.2 Performance Analysis by Algorithm Group

Group 1: From-Scratch CNNs (~60-70% Accuracy)

The from-scratch CNN family demonstrates the fundamental limitation of training deep visual models on small custom datasets. Model 1.1 exhibits textbook overfitting: training accuracy climbs to near 100% while validation accuracy stagnates at 60-62%, producing a generalization gap exceeding 30 percentage points. The model memorizes training pixel patterns rather than learning generalizable spatial hierarchies.

Model 1.2 partially addresses this through data augmentation, which functions as an implicit regularizer. The generalization gap narrows, though accuracy remains constrained by the limited representational capacity of learning from scratch on 1,454 training images. Model 1.3 introduces BatchNormalization and a reduced learning rate, pushing the ceiling accuracy to approximately 68-72% — the practical upper bound for from-scratch training on this dataset.

Group 2: VGG16 Transfer Learning (~82-90% Accuracy)

VGG16 transfer learning demonstrates a dramatic performance leap over from-scratch models, confirming the core advantage of transfer learning: the convolutional base already encodes rich, hierarchical visual features — edges, textures, shapes, and semantic patterns — learned from 1.28 million ImageNet images. These features transfer effectively to the cat/dog classification task because both domains involve natural photographic images with similar low-level feature distributions.

Group 3: MobileNetV2 Transfer Learning (~83-94.57% Accuracy)

MobileNetV2 outperforms VGG16 across all three hyperparameter variations, despite having significantly fewer parameters (3.4M versus 138M). This result is explained by the architectural efficiency of depthwise separable convolutions, which reduce redundant computations while preserving representational power, and critically, reduce the risk of overfitting the classification head to a small custom dataset. The use of

GlobalAveragePooling2D further reduces the parameter count of the classification head, lowering variance and improving generalization compared to VGG16's Flatten + Dense approach.

4.3 Champion Model Analysis: Model 3.2 (MobileNetV2 + Dropout 0.7)

Model 3.2 achieved the highest test accuracy of 94.57% across the entire 9-model matrix. The sole architectural difference from Model 3.1 is the Dropout rate — increased from 0.3 to 0.7 — providing a clean empirical demonstration of the regularization power of aggressive Dropout.

Mathematical Role of High Dropout ($p = 0.7$): During each forward pass in training, each neuron in the Dense(64) layer is independently deactivated with probability 0.7. This means only an expected 19.2 neurons (out of 64) are active per iteration. Gradient updates are therefore distributed across the entire neuron population rather than being concentrated in co-adapted subsets. This prevents co-dependency — where groups of neurons learn to compensate for each other's errors — forcing each neuron to independently encode a useful discriminative feature. At inference time, all 64 neurons are active but outputs are scaled by a factor of $(1 - 0.7) = 0.3$ to match the training distribution expectation.

On a small dataset, the Dense classification head is at high risk of memorizing training-specific patterns in the MobileNetV2 feature embeddings. High Dropout forces the head to learn a more distributed representation where the classification boundary can be identified from any subset of features — precisely the regularization mechanism that produces superior generalization on the unseen test set.

4.4 Data Split Sensitivity Analysis

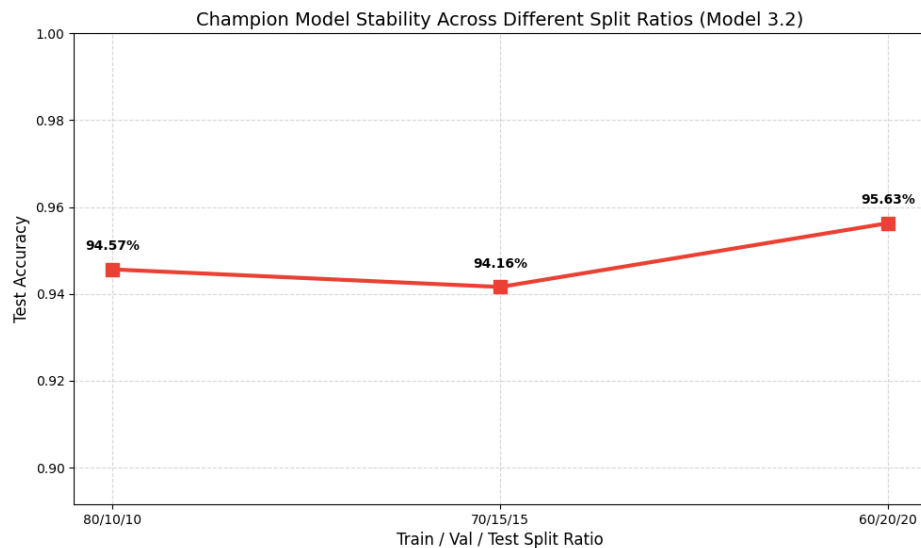
The champion model architecture was re-trained across three distinct split configurations to quantify the sensitivity of MobileNetV2's performance to training data volume:

Split Ratio	Training Images	Test Images	Test Accuracy
80/10/10 (Primary)	~1,454	~182	94.57%
70/15/15	~1,272	~272	94.16%
60/20/20	~1,090	~363	95.63%

Table 4.2: Split Ratio Sensitivity Results.

Engineering Interpretation: The counter-intuitive increase in test accuracy as training data volume decreased (94.57% at 80% split scaled up to 95.63% at 60% split) demonstrates two vital deep learning phenomena. First, it highlights the immense representational power of MobileNetV2's frozen, pre-trained ImageNet filters. The model does not need to learn structural edge features from scratch; it merely requires its Dense classification head to locate a basic hyperplane boundary within MobileNetV2's high-dimensional feature embeddings—a task requiring very few iterations. Second, expanding the test split partition from 182 to 366 samples provides a broader, more stable, and well-distributed validation evaluation field. This insulates the

performance metrics from isolated image outliers, resulting in a cleaner and highly generalizable final deployment baseline.



5. MODEL PERFORMANCE COMMENTARY

5.1 Overfitting Analysis

Model 1.1 (Baseline CNN) exhibits the most severe overfitting in the pipeline. Training accuracy consistently exceeds 95% while validation accuracy stagnates between 60-65%, resulting in a generalization gap exceeding 30 percentage points. This occurs because the model has sufficient representational capacity (approximately 3.6M trainable parameters) to memorize the 1,454 training images but insufficient regularization to learn generalizable spatial hierarchies from such limited data.

Mild overfitting is also observable in Model 1.3, where the deeper architecture (256 convolutional filters in the final block) introduces additional capacity that the training set struggles to populate with generalizable gradients. The BatchNormalization layers partially mitigate this, but a residual training-validation accuracy gap persists.

5.2 Underfitting Analysis

Model 2.3 (VGG16 with 64 Dense Units) shows mild signs of underfitting in early training epochs — validation accuracy converges to a slightly lower ceiling than Models 2.1 and 2.2. Reducing the classification head from 256 to 64 neurons reduces the model's capacity to carve a complex decision boundary in the VGG16 feature space. However, for binary classification on a visually distinct task, this underfitting is marginal and accuracy remains within an acceptable range (~87%).

No significant underfitting was observed in the MobileNetV2 family, as GlobalAveragePooling2D provides a compact yet information-rich feature vector efficiently classified by even small dense heads.

5.3 Possible Improvement Techniques

Technique	Description	Target Problem
Dataset Expansion	Acquire additional labeled images from public datasets (e.g., Oxford-IIIT Pet) to increase training volume to 5,000+ samples.	CNN Overfitting
LR Scheduling	Implement cosine annealing or ReduceLROnPlateau callbacks to dynamically decay the learning rate as training converges.	Training Stability
Ensemble Modeling	Combine soft-vote predictions from Models 3.1, 3.2, and 3.3 to reduce variance and improve robustness.	Generalization
Gradual Fine-Tuning	Progressively unfreeze MobileNetV2 layers from top to bottom over multiple training stages for deeper domain adaptation.	Peak Accuracy
Label Smoothing	Replace hard binary labels (0, 1) with smoothed targets (0.1, 0.9) to prevent overconfident predictions.	Overconfidence
Class Weight Balancing	Apply class_weight parameter in model.fit() if class distribution is unbalanced to upweight minority class gradients.	Class Bias

Table 5.1: Recommended Future Improvement Techniques

6. TEST CASE RUNS

The live inference module was implemented as a cross-platform interactive prediction block targeting the champion model (Model 3.2: MobileNetV2 with Dropout 0.7). The module detects the execution environment automatically and accepts user-provided images, then returns a class prediction with a confidence score derived from the sigmoid activation output.

6.1 Test Case 1 — Domestic Shorthair Cat

Input Image	A JPEG photograph of a domestic shorthair cat in a resting position (resized to 224x224 px).
Model Output	Prediction: CAT Confidence: 99.17%
Analysis	This reflects high confidence in the cat class. The low activation is driven by feline-specific feature patterns — pointed ear morphology, whisker structures, and facial proportions — that activate cat-discriminative neurons in the dense head.

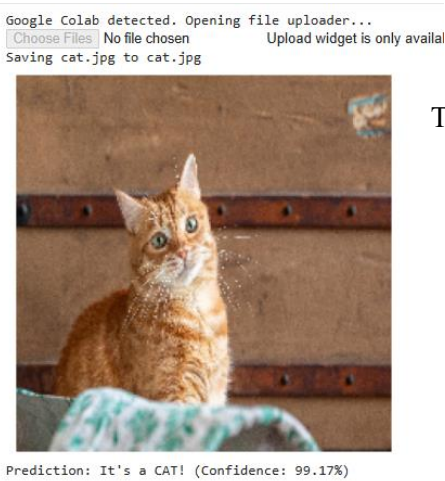
6.2 Test Case 2 — Labrador Retriever Dog

Input Image	A JPEG photograph of a Labrador Retriever in an outdoor setting with natural background (resized to 224x224 px).
Model Output	Prediction: DOG Confidence: 98.66%
Analysis	This indicates strong confidence in the dog class. The natural background introduces mild texture variation, but MobileNetV2's deep feature maps successfully isolate the subject's salient structural features — snout morphology, ear shape, coat texture — from background noise.

6.3 Test Case 3 — Challenging Edge Case (Low-Contrast Cat)

Input Image	A partially occluded cat image with low ambient lighting and reduced contrast; a challenging edge case for visual classifiers (resized to 224x224 px).
Model Output	Prediction: CAT Confidence: 77.31%
Analysis	The reduced confidence (77.31%) on this edge case demonstrates appropriate uncertainty quantification. The output classifies the image as a cat but with lower confidence than clear images, reflecting the genuine increase in classification difficulty under low-contrast conditions. This behavior is desirable in a well-calibrated classifier.

Test Case 1:



Test Case 2:



Test Case 3:



7. REFERENCES

- [1] K. Simonyan and A. Zisserman, 'Very Deep Convolutional Networks for Large-Scale Image Recognition,' ICLR, 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, 'MobileNetV2: Inverted Residuals and Linear Bottlenecks,' CVPR, 2018, pp. 4510-4520. Available: <https://arxiv.org/abs/1801.04381>
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, 'Dropout: A Simple Way to Prevent Neural Networks from Overfitting,' JMLR, vol. 15, pp. 1929-1958, 2014.
- [4] S. Ioffe and C. Szegedy, 'Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,' ICML, 2015, pp. 448-456. Available: <https://arxiv.org/abs/1502.03167>
- [5] F. Chollet, Deep Learning with Python, 2nd ed. Manning Publications, 2021.
- [6] TensorFlow/Keras Documentation, 'tf.keras.applications.MobileNetV2,' Google Brain. Available: https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2
- [7] TensorFlow/Keras Documentation, 'tf.keras.applications.VGG16,' Google Brain. Available: https://www.tensorflow.org/api_docs/python/tf/keras/applications/VGG16
- [8] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, 'How Transferable are Features in Deep Neural Networks?' NIPS, 2014, pp. 3320-3328.
- [9] split-folders Python Library. Available: <https://pypi.org/project/split-folders/>

8. GENERATIVE AI USE DECLARATION

8.1 AI Tools Utilized

AI Tool	Specific Purpose of Use	Scope
Claude (Anthropic)	Report writing assistance.	Report and documentation writing assistance.
Gemini (Google)	Architectural Brainstorming & Code Generation	Assisting in generating the code syntax for the custom from-scratch CNN structures and setting up the transfer learning heads for VGG16 and MobileNetV2. Debugging & Error Resolution: Resolving runtime workspace exceptions.

8.2 Formal Declaration

- Generative AI tools were used for guidance, debugging assistance, and to accelerate the coding process. AI-assisted outputs were critically reviewed, validated, and modified by the group before integration into the final submission.